Software Requirements Specification (SRS) OF CONNECTSPHERE

1. Introduction

The software requirements for a social media like application, developed using Python and Tkinter for the graphical user interface (GUI). The system allows users to manage their profiles, follow other users, create posts, comment on posts, like/unlike posts, and update their bios.

2. Overall Description

The system is designed to function as a social platform where users can interact by following others, posting content, and engaging with posts through comments and likes. It consists of three major components:

- **User Management**: Manages user registration, authentication, bio updates, and password changes.
- **Post Management**: Handles the creation, viewing, liking, commenting, and deletion of posts.
- **Session Management**: Manages user sessions, logging users in and out, and tracking the current session.

3. Functional Requirements

3.1 User Management

- **create_user**: Allows the creation of a new user with a username and password. The password is stored securely in a file.
- **authenticate_user**: Verifies user credentials to allow access.
- **follow_user**: Enables users to follow others by appending the followed user’s username to a follow file.
- **get_followed_users**: Retrieves a list of all users that the current user follows.
- **get_user_bio**: Fetches the user’s bio.
- **update_user_bio**: Allows users to update their bio.
- **change_password**: Enables users to change their passwords securely.

3.2 Post Management

- **get_posts**: Retrieves all posts from the system and displays them in a list format.
- **create_post**: Allows users to create new posts with a title, content, and optional image.
- **like_post**: Increases the like count for a post.
- **unlike_post**: Decreases the like count for a post, ensuring it doesn't go below zero.
- **delete_post**: Deletes a post by its ID.
- **update_post**: Updates the content or title of an existing post.
- **save_posts**: Saves all posts to the system's storage.
- **get_post_by_id**: Retrieves a specific post by its ID.
- **share_post**: Simulates sharing a post.
- **comment_post**: Allows users to comment on a post.
- **get_comments**: Retrieves and formats all comments on a specific post.

3.3 Session Management

- **start_session**: Authenticates the user and starts a session by creating a session file.

- **end_session**: Ends the current session and deletes the session file.
- **get_current_user**: Retrieves the username of the current session.

4. Non-Functional Requirements

4.1 Performance Requirements

- The system should handle multiple users and posts efficiently, with low latency for user interactions (e.g., following, liking, commenting).

4.2 Usability Requirements

- The user interface should be simple, intuitive, and accessible. Users should be able to:
    - Create accounts, log in, and log out with ease.
    - Update profiles and interact with posts in a user friendly manner.

4.3 Reliability Requirements

- The system should be stable, with minimal downtime.
- Data should be backed up to avoid loss of information, especially regarding posts and user details.

4.4 Security Requirements

- User passwords should be stored securely in text files.
- Session management should be secure, with proper login authentication.
- User data should be protected from unauthorized access.

5. External Interface Requirements

5.1 User Interface

The user interface is developed using **Tkinter**. The GUI will:

- Allow users to interact with the system through buttons, text fields, and menus.
- Display posts in a scrollable list with options to like, comment, and share.
- Provide profile management options, including updating bios and changing passwords.

5.2 Hardware Interface

The system does not require any specific hardware beyond the host computer running Python and Tkinter.

5.3 Software Interface

The system will interface with the Python standard library and Tkinter for GUI operations. It will also rely on simple file handling for storing user data, posts, and session information.

6. System Architecture

The system follows a modular architecture with separate components for:

- **User Management**: Handles user data and authentication.
- **Post Management**: Handles post creation, interaction, and deletion.
- **Session Management**: Manages user sessions and tracks the current logged in user.

Data is stored in text files, including user data, posts, and session information. The GUI is implemented using Tkinter, providing an interface for users to interact with the system.

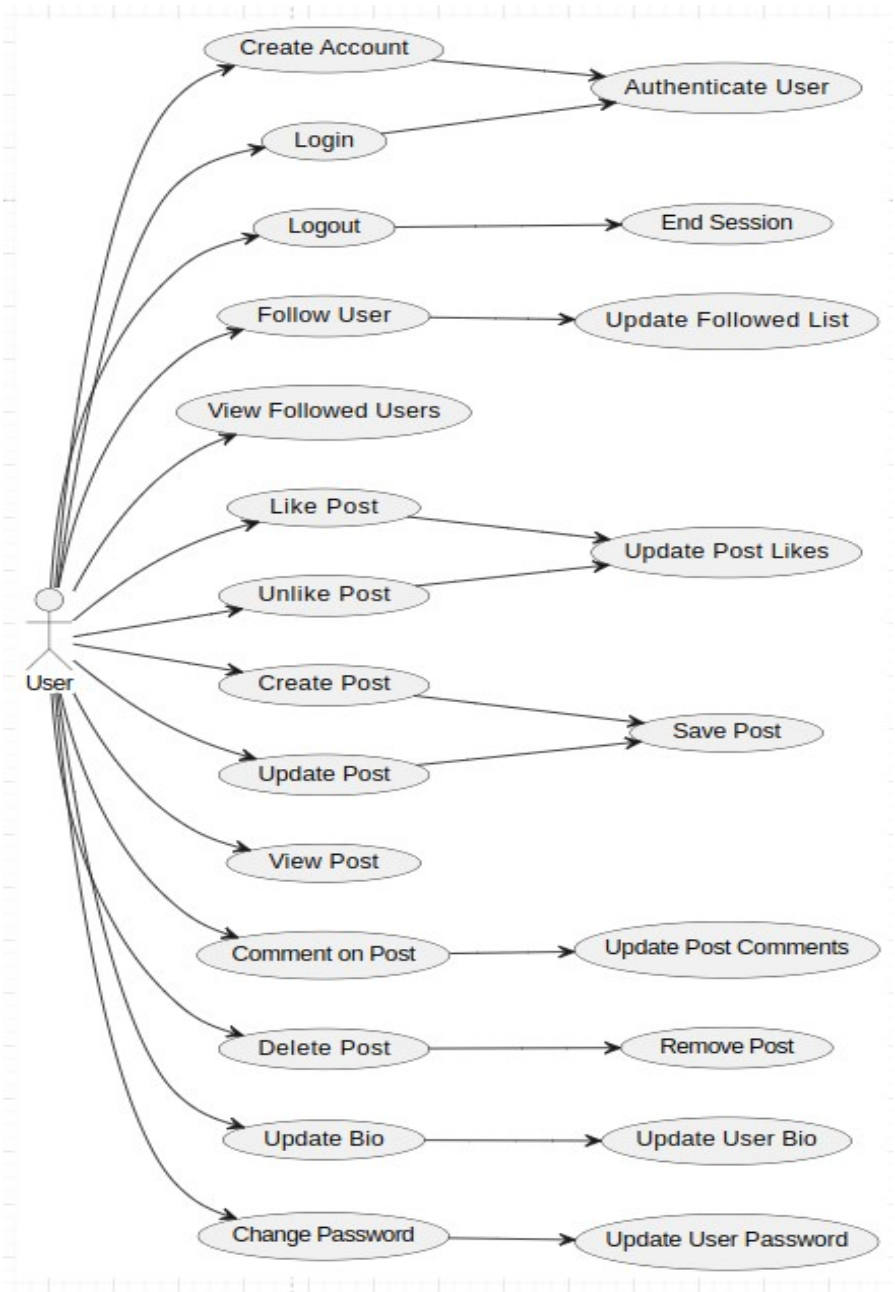
7. System Design

- **Database Design**: Data is stored in text files in a directory named user_data:
  - Each user has a password file (<username>.txt).
  - User bios are stored in _<username>_bio.txt.
  - User follows are stored in _<username>_follow.txt.
  - Posts are stored in posts.txt in the format: username|title|content|timestamp|likes|comments|category|image.

8. Use Case Diagram

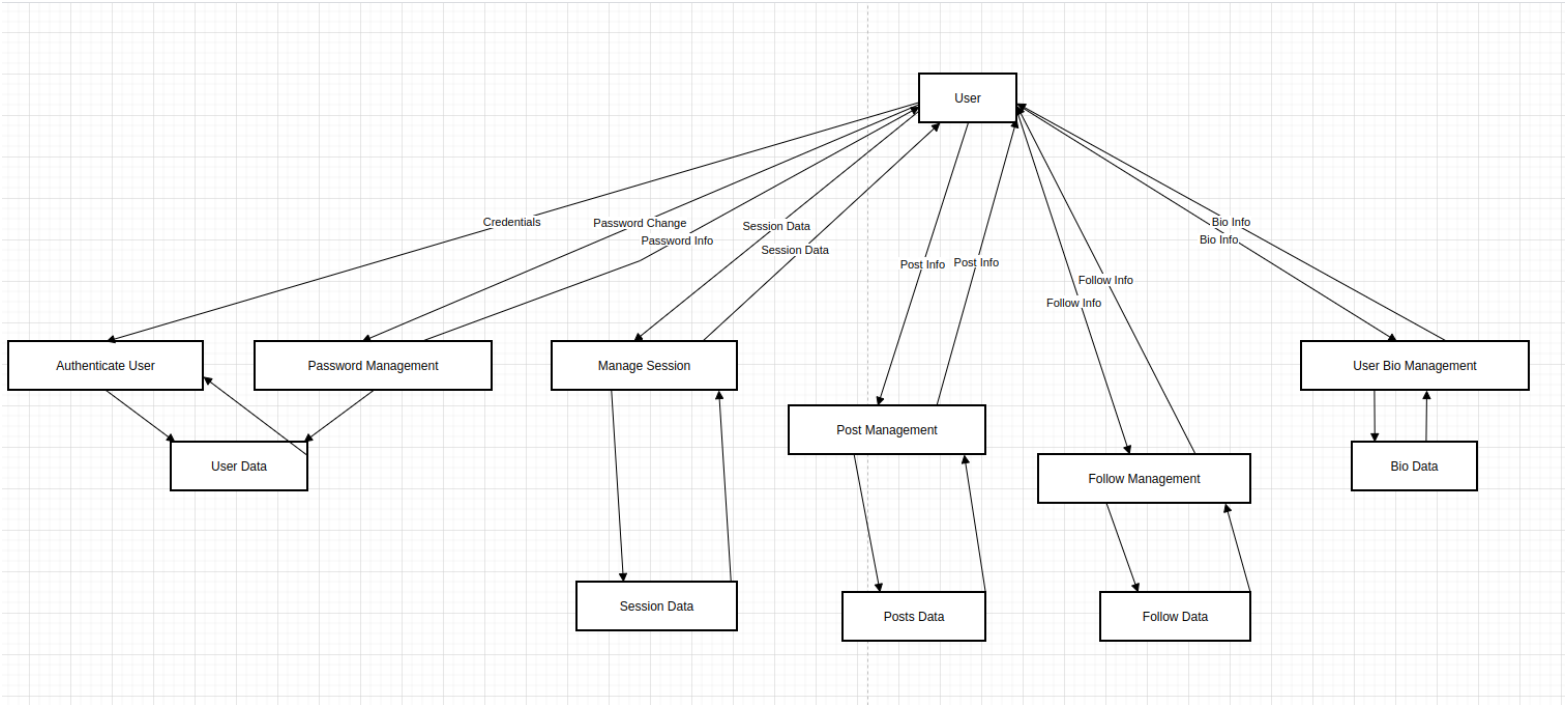
The use case diagram would include the following actors and actions:

- **User**: Create account, login, logout, follow other users, create and manage posts, comment, like/unlike posts, update profile, and change password.



9. Data Flow Diagram

A data flow diagram (DFD) would depict how data flows between the user interface, session manager, user manager, post manager, and data storage (text files).



10. Class Diagram:

